



6

ENUMERATING HARDWARE AND NETWORK CONFIGURATIONS



Hardware information, such as CPU speed and RAM allotment, and networking configurations, such as the MAC and IP addresses of the host's network interfaces, can indicate to malware that it's operating in a lab environment. Additionally, malware can use these pieces of information to establish context within its operating environment. In this chapter, we'll discuss the techniques that malware might use to gather this information and evade detection.

Hardware and Device Configurations

System hardware configurations can provide valuable information to a malware sample that it can use to determine whether it's running in a VM or sandbox. VMs use emulated hardware that can easily be differentiated from real hardware, and they are often configured much differently from physical systems. The malware can also enumerate hardware information, like the number of processors, the amount of

RAM installed in the system, the hard drive storage capacity, and more. In this section, we'll explore each of these areas in turn.

CPU

Because virtualized CPUs look a bit different from their physical counterparts, malware will often check the host's processor to determine the context in which it's running. There are a few ways the malware can retrieve CPU hardware information. One of the more common approaches is calling the `GetSystemInfo` function to get the `dwNumberOfProcessors` value, which represents how many processors the system has. Modern computer systems almost always have more than one processor and processor core. To determine whether it's running on a virtual machine, the malware might check whether the host has fewer than two processor cores, like so:

```
SYSTEM_INFO systemInfo;  
GetSystemInfo(systemInfo);  
int numProcessors = systemInfo.dwNumberOfProcessors;  
if numProcessors < 2 {  
    KillSelf()  
}
```

In this pseudocode, the sample defines a struct (`systemInfo`), then calls `GetSystemInfo`. A *struct*, short for structure, is a data type that allows for grouping of other data under a single name. All of the information returned from the `GetSystemInfo` function will be stored in the `systemInfo` struct. The malware then inspects the `dwNumberOfProcessors` value in the struct and, if the `numProcessors` value is less than 2, will terminate itself.

Similarly, a malware sample can call the following functions to return information it can use to infer the environment it's running in:

GetLogicalProcessorInformation Returns processor core information.

GetNativeSystemInfo Returns similar information as `GetSystemInfo`, but is normally invoked by 64-bit applications.

IsProcessorFeaturePresent Returns the status of various processor features. If it returns true for `PF_VIRT_FIRMWARE_ENABLED`, the system is using virtual firmware, an obvious clue that the system is likely virtualized.

The Process Environment Block can also be used for CPU enumeration. To recap from [Chapter 1](#), the PEB structure consists of pointers to other structures in memory that contain information about the currently running process. The following code shows how malware can search the PEB to retrieve CPU information:

```
mov eax, dword ptr fs:[0x30]
mov edx, dword ptr ds:[eax+0x64]
cmp edx, 1
je terminate_process
```

The sample moves the address of the PEB (stored in `fs:[0x30]`) into the `eax` register, then puts the specific offset of the PEB in which the malware is interested (`[eax+0x64]`) in the `edx` register. Offset 0x64 in the PEB structure stores the number of processor cores configured on the system. If the number of processor cores is 1, the malware terminates itself by jumping to the `terminate_process` function.

NOTE

For 64-bit processes, the address of the PEB will be in the `gs` register at `gs:[0x60]`, so be on the lookout for malware referencing this address as well.

RAM

Modern computers typically have a minimum of 4GB of RAM, but some malware analysis environments and sandboxes may not have

this amount. If the installed RAM is below the 4GB threshold (or similar), the malware might assume it is running in a virtual environment. To check this, the malware calls the Windows function `GetPhysicallyInstalledSystemMemory` to return the amount of system memory, like so:

```
lea ecx, [ebp+TotalMemoryInKilobytes]
call GetPhysicallyInstalledSystemMemory
cmp ecx, 4194302
jl  terminate_process
```

Here the malware sample calls the `GetPhysicallyInstalledSystemMemory` function with `TotalMemoryInKilobytes` as a parameter, which will store the total system memory value returned from the function. After the function call, the malware compares the `TotalMemoryInKilobytes` value with 4194302, which is 4,194,302 kilobytes, or 4GB. If the amount of system RAM is lower than 4GB, the malware sample will jump to `terminate_process` and delete itself.

Hard Disks

Most modern computers are allotted hundreds of gigabytes of storage space. Malware can query the storage capacity of the hard disks, and if that value is below a certain threshold, it might determine that it's running in a virtual environment. This threshold is typically 40GB, 60GB, or 80GB, but this number may increase as average storage capacity grows.

The most common Windows functions used for gathering hard disk and capacity information are `GetDiskFreeSpace` and `GetDiskFreeSpaceEx`. Other than evasion, there are limited reasons why malware would be querying the hard disk space, so these functions should raise a red flag for you. An easy way to bypass this evasion tactic is to increase the size of your VM disks to more than 40GB or, even better, 80GB.

Windows contains many API functions that are not well known and are less commonly used. One example is the `IsNativeVhdBoot` function:

```
call IsNativeVhdBoot
test eax, 0
jne loc_403DDD
```

If `IsNativeVhdBoot` doesn't return 0, the system was booted from a virtual hard disk, which indicates a VM. Note that the `IsNativeVhdBoot` function is effective only on Windows 8 and above.

Monitor Configurations

A typical modern computer setup will likely have a high-resolution monitor and sometimes more than one, whereas (most) VMs and sandboxes will not. Some malware specifically looks for the number of monitors in use, and if it detects, say, only one monitor, it might infer that it's in an analysis environment. To accomplish this, it can call the Windows API function `GetSystemMetrics`, which returns a specific variable called `SM_CMONITORS` representing the number of monitors in use. Another option is to call the `EnumDisplayMonitors` function, which doesn't return the total number directly but instead executes a callback function for each monitor. A *callback function* is simply a way of performing tasks for each object—in this case, a monitor—that invokes it.

Malware can also check screen resolution using the same functions. The typical resolution for a non-virtualized end-user system is quite high, such as $1,600 \times 900$. If the display resolution of your analysis machine is lower (say, $1,152 \times 864$), malware might note this.

THE GETSYSTEMMETRICS FUNCTION

`GetSystemMetrics` returns a lot of useful information that malware can take advantage of, both for VM and sandbox detection and for general system

enumeration. Here's just a sampling of information it returns:

SM_CMONITORS The number of display monitors configured

SM_CMOUSEBUTTONS The number of mouse buttons (returns 0 if no mouse is installed)

SM_CXSCREEN The pixel width of the screen

SM_CYSCREEN The pixel height of the screen

SM_MIDEASTENABLED Middle Eastern language configuration (returns a non-zero value if the system is configured for Hebrew or Arabic)

SM_MOUSEPRESENT The presence of a mouse (returns 0 if one isn't installed)

SM_NETWORK The presence of a network (if present, the least significant bit is set; if not, bits are cleared)

USB Controllers

Many VMs are configured to use either an older USB protocol version (V1.1 or V2.0, for example) or no USB controller device at all. Because most modern non-virtualized systems have at least one USB controller with an up-to-date USB device version, this can be a good clue for malware.

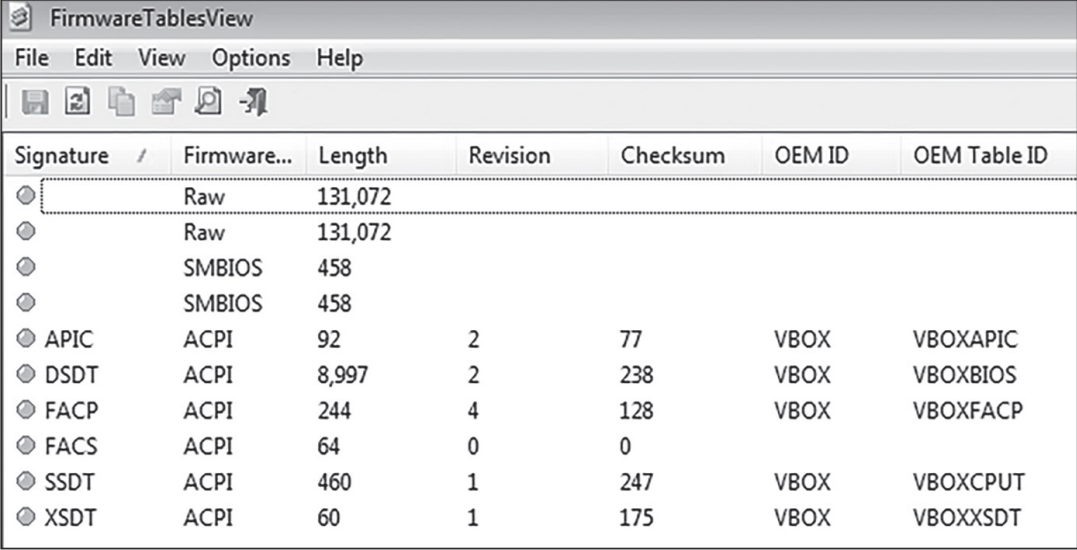
Windows provides a helpful API just for enumeration of USB devices and settings: *Winusb.dll*. If you spot malware attempting to import this DLL and use its functions, it's a strong indicator that something fishy is going on.

Firmware Tables

Most system hardware is accompanied by low-level software known as *firmware*. Firmware breathes life into the hardware; without it, the hardware couldn't interface with the OS or any other programs.

Windows keeps track of firmware in *firmware tables* that also contain the make and model of the associated hardware, which malware can exploit to identify any hypervisor-related information.

Let's take a look at a firmware table. **Figure 6-1** shows the output on my VM of Nirsoft's free tool FirmwareTablesView.



The screenshot shows the FirmwareTablesView application window. It has a menu bar with File, Edit, View, Options, and Help. Below the menu bar is a toolbar with icons for opening, saving, and other file operations. The main area is a table with the following columns: Signature, Firmware..., Length, Revision, Checksum, OEM ID, and OEM Table ID. The table contains several rows of data, including two 'Raw' entries and several ACPI tables (APIC, DSDT, FACP, FACS, SSDT, XSDT) with their respective lengths, revisions, checksums, and OEM IDs (all VBOX).

Signature	Firmware...	Length	Revision	Checksum	OEM ID	OEM Table ID
Raw	Raw	131,072				
Raw	Raw	131,072				
SMBIOS	SMBIOS	458				
SMBIOS	SMBIOS	458				
APIC	ACPI	92	2	77	VBOX	VBOXAPIC
DSDT	ACPI	8,997	2	238	VBOX	VBOXBIOS
FACP	ACPI	244	4	128	VBOX	VBOXFACP
FACS	ACPI	64	0	0		
SSDT	ACPI	460	1	247	VBOX	VBOXCPUT
XSDT	ACPI	60	1	175	VBOX	VBOXXSDT

Figure 6-1: Firmware tables displayed in FirmwareTablesView

You can likely already spot the anomalies that a malware sample could take advantage of. There are multiple references to *VBOX*, which is the standard prefix for VirtualBox hypervisor firmware.

In the Firmware column, you can see the type of each firmware table, also called the *provider signature*: ACPI, SMBIOS, or Raw. Some functions, like `GetSystemFirmwareTable` and `EnumSystemFirmwareTables`, require a provider signature to retrieve firmware tables. The following code shows a piece of malware calling `EnumSystemFirmwareTables` to inspect the ACPI tables:

```
loc_10001300:  
push  [esp+38h+BufferSize] ; BufferSize  
push  esi                    ; pFirmwareTableEnumBuffer  
push  'ACPI'                 ; FirmwareTableProviderSignature  
call  EnumSystemFirmwareTables  
cmp   [esp+38h+BufferSize], eax
```

Later, this malware will search the buffer storing the firmware tables for hypervisor-related strings like Oracle, Vbox, VirtualBox, VMware, and VMware, Inc.

The `NtQuerySystemInformation` function also returns a lot of different system information, for both malicious and legitimate reasons. Malware can use this function to enumerate firmware tables. Here's what this might look like:

```
push  [ebp+Length] ; SystemInformationLength  
push  eax          ; SystemFirmwareTableInformation  
push  76           ; SystemInformationClass  
call  NtQuerySystemInformation
```

The malware calls the `NtQuerySystemInformation` function with a few important parameters. The first parameter, `SystemInformationLength` (`[ebp+Length]`), is the size of the buffer that will receive the data returned from the function call. The second parameter, `SystemFirmwareTableInformation` (`eax`), is a pointer to that buffer. The third parameter, `76`, represents the system information class that this malware is interested in, `SystemFirmwareTableInformation`. After the malware calls the `NtQuerySystemInformation` function and the firmware table structure is stored in the buffer, it can enumerate the buffer for hypervisor-related information.

Note that the `SystemFirmwareTableInformation` data class is undocumented by Microsoft, so there isn't much public information about it. Microsoft keeps certain functionalities internal, likely so that they

can't be easily abused, but malware authors and researchers will inevitably find them. Conix Cybersécurité has compiled a complete list of the data classes that `NtQuerySystemInformation` can return at https://github.com/conix-security/zer0m0n/blob/master/src/driver/include/nt/structures/SYSTEM_INFORMATION_CLASS.h.

Other Hardware Devices

Examples of other hardware configurations that malware might use to obtain information about its environment are CD or DVD drives, sound cards and audio configurations, attached printers (or the lack thereof), and serial ports. We'll go over some more of these settings in [Appendix A](#), but because malware could enumerate any and all hardware configurations on the victim system, it's impossible to include all of them in this book.

Next, we'll turn our attention to the networking-related information that malware might use to suss out a VM or sandbox environment.

Networking-Related Artifacts

Networking-related artifacts such as IP address configurations, currently established connections, open service ports, and even the domain that the victim system is joined to can help malware detect its operating environment. In this section, you'll learn how.

IP Address Configurations

Malware may want to get the host's IP address for a number of reasons, most notably to detect if the host is running in a sandbox or malware analysis lab. The default IP address range for internal VirtualBox networks is 192.168.56.X. The default IP range for VMware Workstation is 192.168.X.X, with the last two octets being randomized (such as 192.168.5.100, or 192.168.187.101). The range depends on what version of the hypervisor software the system is running. If the

victim system has an IP address in these ranges, the malware might either determine it's a VM or further scrutinize the system.

Retrieving the IP address of the host is as simple as leveraging the `GetAdaptersAddresses` function:

```
push edx ; AdapterAddresses structure
push 0   ; Reserved parameter
push 0   ; Flags parameter
push 2   ; Family parameter
call GetAdaptersAddresses
--snip--
mov eax, [ebp+AdapterAddresses.FirstUnicastAddress]
mov edx, [ebp+bad_ip_address]
cmp eax, edx
jnz terminate_process
```

Like many Windows API functions, `GetAdaptersAddresses` takes a series of parameters. In this case, `push edx` pushes to the stack the address of the buffer `AdapterAddresses`, which will hold all the address data returned from the function call. The next three instructions push the `reserved`, `flags`, and `family` parameters. The most important is the `family` parameter, which contains a value of 2, telling `GetAdaptersAddresses` to return only IPv4 information. [Table 6-1](#) lists the definitions of each `family` parameter value.

Table 6-1: `GetAdaptersAddresses` family Parameter Values

ASCII value	Numerical value	Definition
AF_UNSPEC	0	Returns both IPv4 and IPv6 addresses
AF_INET	2	Returns only IPv4 addresses

ASCII value	Numerical value	Definition
AF_INET6	23	Returns only IPv6 addresses

Next, the malware calls the `GetAdaptersAddresses` function. Later in the code, the value of `AdapterAddresses.FirstUnicastAddress` is moved into `eax`, and the variable `bad_ip_address` is moved into `edx`. The `FirstUnicastAddress` variable is part of `AdapterAddresses` and contains the first IP address in the structure. The `bad_ip_address` variable contains the IP address that the malware is checking against. Let's assume this value is `192.168.56.2`, indicating a VirtualBox network. The malware compares the `FirstUnicastAddress` value with the `bad_ip_address` using `cmp eax, edx` to see if the victim system has the IP address `192.168.56.2`. If this IP exists in the `AdapterAddresses` structure, the malware jumps to the `terminate_process` function to terminate itself.

Keep in mind that there are many ways to enumerate the IP addresses on a system. Some functions involve querying the interface information directly, while others use a more back-channel method, such as listing the IP network tables. IP address information can also be queried using Windows tools such as WMI, PowerShell, and the Windows command line. You should be suspicious of any malware looking for the host's internal IP address, as it could be using a detection technique.

Domain Configurations

Malware may attempt to enumerate the victim host's domain to ensure the target is within a certain company or network, or to weed out malware analysis environments. In this case, a *domain* is simply a logical grouping of systems on a network. The computer you use for work is likely part of a domain whose name relates to your company name. This domain is also typically part of your system's *hostname*, or the

name of your device on the network. If you work at Evil Corp, for example, your domain name might be *evil.corp* and your hostname might be *your.computer.evil.corp*.

Domain enumeration can be a great evasive tactic for malware. For example, if the victim system isn't associated with a domain, or if the domain it's attached to doesn't match the malware's primary target, the malware may terminate itself to avoid analysis or change its behavior. Common Windows API functions for enumerating a host's domain are `GetComputerName` and `DsGetDcName`.

In addition to Windows API functions, malware may query these registry keys, which may contain the domain of the system:

- HKCU\Volatile Environment
- HKLM\System\CurrentControlSet\Services\Tcpip\Parameters

To thwart this tactic, join your VMs and sandboxes to a domain. Even if the domain is fake, it can still trick malware into executing. We'll discuss more about faking domains and network connectivity in [Appendix A](#).

MAC Address Configurations

A *media access control (MAC) address* is a unique identifier applied to all computer network hardware. The MAC address consists of digits (3 bytes) that represent the device manufacturer, followed by another series of digits (3 additional bytes) specific to the host's hardware devices (see [Figure 6-2](#)).

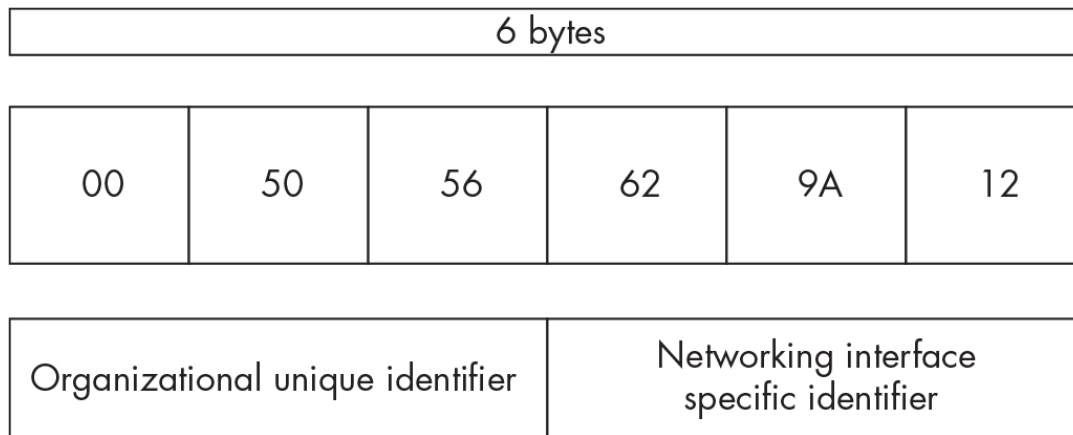


Figure 6-2: MAC address structure

In **Figure 6-2**, the example MAC address is 00:50:56:62:9A:12. The first 3 bytes (00:50:56) represent the manufacturer's identifier (in this case, VMware), and the last 3 bytes represent this specific adapter.

Malware can query the host's MAC addresses to see if the host is using a specific manufacturer's network hardware. Each piece of network hardware has its own dedicated MAC address as well, so malware can also use this information to target its victims. An example of this is the now well-known supply chain attack dubbed "ShadowHammer," which occurred in 2019. The attackers used a list of preselected MAC addresses to target and infect specific computers.

To obtain a list of MAC addresses on the host, malware can call the Windows API function `GetAdaptersAddresses`. It then compares that list to a hardcoded list of MAC addresses, infecting the system only if it finds a network device with a matching MAC address. After the malware calls `GetAdaptersAddresses` (using the same technique I outlined in "IP Address Configurations" on [page 109](#)), it checks the `PhysicalAddress` value of the `AdapterAddresses` structure, which contains the host's MAC addresses, like so:

```
call GetAdaptersAddresses
--snip--
mov eax, [ebp+AdapterAddresses.PhysicalAddress]
--snip--
```

```
mov [ebp+var_38], 0B203B000h ; MAC address data
mov [ebp+var_34], 0F6DDB6CAh ; MAC address data
mov [ebp+var_30], 1D3CA6CDh  ; MAC address data
mov [ebp+var_2C], 287E2CDBh  ; MAC address data
```

Later, the malware moves data onto the stack with instructions like `mov [ebp+var_38], 0B203B000h`. This data represents part of the MAC address that the malware will compare to the victim's MAC addresses.

VM-related MAC addresses can also be enumerated this way. The presence of one of the following MAC address prefixes would indicate to the malware that it's running in a VM environment and possibly being analyzed:

VMware Workstation

- 00:50:56 ...
- 00:0C:29 ...
- 00:05:69 ...

VirtualBox

- 08:00:27 ...

NOTE

To bypass this detection technique, change the MAC addresses of your VMs and sandboxes.

External IP Address and Internet Connectivity

One evasion technique that has seen increasing use is obtaining the potential victim's external IP address. Malware might want this information for two reasons. First, malware can use it to determine the potential victim's location. Some malware may target victims only in a certain area of the world, or it might want to exclude certain geo-

graphical regions. Not only does this help the malware stay on target, but it also aids in anti-analysis and evasion; if the potential victim isn't within the malware's regional scope, the malware will terminate itself or modify its behavior, likely fooling sandboxes and malware analysts.

Second, malware can use the host's external IP address information or internet connection status to detect an analysis environment. For example, if the external IP is a popular virtual private network (VPN) gateway (such as NordVPN or ExpressVPN) or a TOR exit node, then the malware may infer that it's being analyzed. (After all, if someone is using TOR or a VPN, they *must* be a malware analyst, right?) In addition, it's often the case that sandboxes and analysis environments are intentionally offline, so if the host has no external IP address at all, the malware might assume it's running in a lab environment.

This technique can be executed in a number of ways, but perhaps the most common is simply sending a web request to an IP lookup service, such as *WhatIsMyIP.com* or *IPLocation.net*. These are legitimate services that report back the external IP and geolocation information of a calling host. If you detonate malware in an analysis sandbox or lab and you see HTTP requests to sites like these, it's a signal that the malware might be attempting to look up the host's external IP address or location. Here's how that might look:

```
❶ mov    ecx, [ebp+lpszServerName] ; "icanhazip.com"
   push  0      ; dwContext
   push  0      ; dwFlags
   push  3      ; dwService
   push  0      ; lpszPassword
   push  0      ; lpszUserName
❷ push  80     ; nServerPort
❸ push  ecx    ; lpszServerName
   push  ebx    ; hInternet
❹ call  ds:InternetConnectA
```

First, the malware sample moves the value `lpzServerName`, which contains the domain *icanhazip.com*, into the `ecx` register for later use ❶.

The `InternetConnectA` function takes a lot of parameters, but the important ones are `nServerPort` ❷, which indicates the port on which this malware sample will contact the domain (port 80, or HTTP), and the domain itself, now stored in `ecx` ❸. Finally, it calls `InternetConnectA` ❹. This sample's code is essentially contacting the *icanhazip.com* domain over HTTP to obtain the host's external IP address.

A related method is to simply send a DNS or HTTP request to an online service provider. Any service provider will do, but *Google.com* is a common one. If the malware sample sends a request to a web server and receives no data (or incorrect data), it might deduce that it's being analyzed. The following sample attempts to contact *Google.com* and then checks the response to ensure it matches the normal response from the server:

```
push    ecx      ; lpzServerName ("google.com")
push    ebx      ; hInternet
call    InternetConnectA
--snip--
push    eax      ; lpdwNumberOfBytesRead
❶ push    9        ; dwNumberOfBytesToRead
lea     ecx,     [ebp+lpBuffer]
❷ push    ecx      ; lpBuffer
push    edi      ; hFile
❸ call    InternetReadFile
test    eax, eax
jz      short loc_4021B6
--snip--
loc_404194:
❹ mov     eax, offset first_bytes ; "<!doctype"
test    eax, [ebp+lpBuffer]
```

After attempting to contact *Google.com* (using `InternetConnectA`), the sample pushes parameters for the `InternetReadFile` function, including

`dwNumberOfBytesToRead` ❶, which contains the number of bytes to be read from the web server response (9), and `lpBuffer` ❷, which is a pointer to the data `InternetReadFile` will return. Next, the malware sample calls `InternetReadFile` ❸, which reads the first 9 bytes from Google's response (which should be `<!doctype`). Finally, the code compares the `<!doctype` value to the actual response stored in `lpBuffer` ❹. If the responses are different, this malware sample might conclude that the system isn't connected to the internet or is perhaps being analyzed in a sandbox.

It's not uncommon to see this external IP validation technique in modern malware, so be on the lookout for it. If you're analyzing a malware sample and see outbound HTTP or DNS requests to websites such as *WhatIsMyIP.com* or to service providers such as *Google.com* or *AWS.Amazon.com*, there's a chance the malware is using this VM detection technique. Here are some other websites to keep an eye out for:

- *api.ipify.org*
- *checkip.amazonaws.com*
- *checkip.dyndns.com*
- *icanhazip.com*
- *ip-api.com*
- *ip.anysrc.net*
- *ipecho.net*
- *ipinfo.io*
- *iplocation.net*
- *myexternalip.com*
- *myipaddress.com*
- *showipaddress.com*
- *whatismyip.com*
- *whatismyipaddress.com*
- *wtfismyip.com*

TCP Connection States

In the previous section, I mentioned that malware analysts often configure their analysis machines and sandboxes to be completely offline or connected to a fake network. Malware can take advantage of this behavior by enumerating the outgoing TCP connections and trying to determine if they’re valid. A legitimate end-user system or server will likely have many outgoing TCP connections to various external IP addresses and ports. A malware analysis machine or sandbox, unless connected to a real network like the internet, may have only a few connections or none at all.

This tactic involves using the Windows API `GetTcpTable` function, which returns a table listing the current status of the latest TCP connections on the system, similar to the result of running `netstat.exe` on the host. This table might look something like [Table 6-2](#).

Table 6-2: Sample TCP Connections

Local address	Local port	Remote address	Remote port	State
127.0.0.1	2869	local-machine	49202	TIME_WAIT
127.0.0.1	2869	local-machine	49203	ESTABLISHED
192.168.1.2	49157	91.184.220.29	80	ESTABLISHED
192.168.1.2	49158	64.233.160.15	443	ESTABLISHED
...	...	...	...	...

An offline malware analysis machine or sandbox likely won’t have any established remote TCP connections like the third and fourth rows

in **Table 6-2**.

Malware can also, of course, leverage native Windows tools for this, such as the aforementioned *netstat.exe*. When inspecting malware, watch for attempts to enumerate TCP state information via Windows functions or native Windows tools.

OPEN PORTS

Certain sandboxes have open ports that are typically used for communication between the hypervisor and the VM that detonates the malware. For example, the open source sandbox Cuckoo opens TCP port 8000 by default for communication.

In order to enumerate these open ports, malware can call the functions described previously, such as `GetTcpTable`. If the malware sees that port 8000 is in a `LISTENING` or `ESTABLISHED` state, the malware might assume it's running in a Cuckoo sandbox environment. More advanced malware may even choose to send data to an open port to see how it behaves, much like a pentester would do to probe a target. Depending on the response from the open port, the malware sample will either continue normally or take evasive actions.

Summary

In this chapter, we covered many of the hardware and network configuration enumeration techniques that malware might use to determine its operating environment. As you've seen, understanding its environment and establishing context are key to malware's ability to detect VMs and sandboxes, evade analysts, or otherwise stay on target. In the next chapter, we'll explore how malware exposes the malware analysis environment by inspecting runtime anomalies, monitoring performance, and abusing virtual processor instructions.

